

Desenvolvimento do Jogo Brick Breaker em Linguagem C Utilizando Raylib

Rafael Silva de Souza , Felipe Portinho de Souza

¹ Universidade Federal do Rio Grande do Sul (UFRGS)
Instituto de Informática

rafesilvadesouza@gmail.com, felipeportinhodesouza@gmail.com

Abstract. *This paper presents the development of Brick Breaker, an arcade game inspired by the classic Breakout. The project was implemented in the C programming language using the Raylib graphics library. Concepts such as modularization, file manipulation, state control, collision detection and graphical interfaces were applied throughout the development process. The final application includes multiple levels, a score system, ranking management and game persistence through save files.*

Resumo. *Este trabalho apresenta o desenvolvimento do jogo Brick Breaker, inspirado no clássico Breakout. O projeto foi implementado em linguagem C utilizando a biblioteca gráfica Raylib. Durante o desenvolvimento foram aplicados conceitos de modularização, manipulação de arquivos, controle de estados, detecção de colisões e interfaces gráficas. A aplicação final inclui múltiplas fases, sistema de pontuação, ranking de jogadores e persistência de dados por meio de arquivos de salvamento.*

1. Introdução

A disciplina de Algoritmos e Programação tem como objetivo desenvolver a capacidade de resolução de problemas por meio da implementação de algoritmos em linguagem C. Como atividade prática final da disciplina, foi proposto o desenvolvimento de um jogo completo contendo interface gráfica, lógica de execução e persistência de dados.

O projeto desenvolvido consiste em um jogo arcade inspirado no clássico Breakout, originalmente lançado pela Atari em 1976. Nesse gênero de jogo, o jogador controla uma plataforma horizontal utilizada para rebater uma bola que deve destruir todos os blocos presentes na fase.

A implementação do projeto permitiu explorar diversos conceitos fundamentais da programação, incluindo estruturas de dados, modularização, manipulação de matrizes, leitura e escrita de arquivos, controle de fluxo, tratamento de eventos e desenvolvimento de interfaces gráficas.

2. Enredo do Jogo

No jogo Brick Breaker, o jogador controla uma plataforma localizada na parte inferior da tela. O objetivo principal é impedir que a bola caia para fora da área de jogo enquanto destrói todos os tijolos presentes em cada fase.

Os blocos possuem diferentes níveis de resistência, representados por valores distintos. Alguns blocos são destruídos imediatamente após uma colisão, enquanto outros exigem múltiplos impactos para serem eliminados. A cada bloco destruído o jogador recebe pontos, aumentando sua pontuação total.

Quando todos os blocos destrutíveis de uma fase são eliminados, o jogador avança automaticamente para a próxima fase. O jogo termina em vitória quando todas as fases disponíveis são concluídas ou em derrota caso o jogador perca todas as vidas disponíveis.

3. Requisitos

Para compilação e execução do projeto são necessários:

- Linguagem C
- GCC (compilador)
- Biblioteca gráfica Raylib
- (Opcional) Doxygen para criação da documentação

4. Estrutura do Projeto

O projeto está organizado nos seguintes diretórios e arquivos principais:

- `bin/` : executável gerado após a compilação
- `fases/` : arquivos de texto contendo os mapas das fases
- `include/` : arquivos de cabeçalho (`.h`)
- `src/` : código-fonte da aplicação (`.c`)
- `saves/` : arquivos binários com progresso das partidas
- `ranking.txt` : armazenamento do ranking de pontuações
- `TODO.md` : controle de tarefas do desenvolvimento
- `Doxyfile` : configuração da documentação
- `README.md` : documentação do projeto

5. Arquitetura da Aplicação

A aplicação foi desenvolvida seguindo princípios de programação estruturada e modularização. O código foi organizado em diferentes módulos independentes, cada um responsável por uma funcionalidade específica do sistema. Essa abordagem permitiu reduzir o acoplamento entre componentes, facilitar a manutenção do código e tornar a implementação mais organizada.

5.1. Estruturas de Dados

A principal estrutura utilizada no projeto é a estrutura `Game`, responsável por armazenar todo o estado atual da partida.

Essa estrutura reúne informações relacionadas à bola, à plataforma controlada pelo jogador, ao sistema de pontuação, ao número de vidas restantes, à fase atual e ao mapa de blocos carregado em memória. Além disso, ela também armazena os dados necessários para integração com o sistema de ranking.

O mapa da fase é representado por uma matriz bidimensional de caracteres. Cada posição da matriz representa um bloco da fase, permitindo identificar rapidamente seu tipo e resistência.

Para o gerenciamento do ranking foi utilizada a estrutura `RankingEntry`, composta pelo nome do jogador e sua respectiva pontuação. Essa estrutura é utilizada para armazenar e organizar os melhores resultados obtidos.

O controle das diferentes telas do jogo é realizado através da enumeração `GameState`, responsável por representar os estados possíveis da aplicação, como menu principal, partida em andamento, ranking, carregamento de jogo salvo, tela de vitória e tela de derrota.

Para as informações referentes foi utilizado uma estrutura contendo o tipo do `powerUp`, tempo de duração, localização na tela, e a ativação dele.

5.2. Organização Modular

O projeto foi dividido em módulos especializados localizados nos diretórios `src` e `include`.

O módulo `main` atua como ponto de entrada da aplicação. Ele é responsável pela inicialização da biblioteca Raylib, criação da janela gráfica e execução do laço principal do jogo.

O módulo `game` concentra toda a lógica da partida. Nesse módulo são implementadas as rotinas de movimentação da bola, movimentação da plataforma, detecção de colisões, controle de vidas, pontuação e progressão entre fases.

O módulo `powerup` implementa a estrutura de `PowerUps` assim como suas funcionalidades, incluindo sorteio, atualização de estado, aplicação de efeitos e interação com a plataforma e demais elementos do jogo.

O módulo `level` é responsável pelo carregamento e renderização das fases. As fases são armazenadas em arquivos texto localizados no diretório `fases`, sendo convertidas para a representação matricial utilizada internamente pelo jogo.

O módulo `menu` implementa as telas de navegação da aplicação, permitindo ao usuário iniciar uma nova partida, carregar um jogo salvo ou visualizar o ranking.

O módulo `ui` é responsável pela renderização da interface do usuário (HUD), incluindo textos, indicadores de vida, pontuação e elementos visuais auxiliares durante o jogo.

O módulo `audio` gerencia a inicialização e reprodução de sons e músicas do jogo, incluindo efeitos sonoros de colisões, eventos e trilha sonora.

O módulo `save` implementa o sistema de persistência de dados. Suas funções permitem armazenar o estado atual da partida em arquivos binários e restaurá-lo posteriormente.

O módulo `ranking` realiza o gerenciamento das maiores pontuações obtidas pelos jogadores. Esse módulo é responsável pela leitura, escrita, ordenação e exibição dos registros armazenados.

5.3. Principais Funções

Dentre as funções implementadas, destacam-se:

- `InitGame`: inicializa uma nova partida configurando os valores iniciais do jogo, incluindo estado da bola, plataforma, nível atual e estruturas auxiliares.

- `UpdateGame`: executa a lógica principal da aplicação, incluindo atualização de estados, processamento de colisões, gerenciamento de power-ups, progressão entre fases e verificação de condições de vitória e derrota.
- `DrawGame`: realiza a renderização dos elementos visuais da partida, incluindo cenário, blocos, bola, plataforma, power-ups e interface do usuário.
- `LoadCurrentLevel`: carrega a fase correspondente ao progresso atual do jogador, inicializando o mapa de blocos e preparando possíveis power-ups do nível.
- `LoadLevel`: realiza a leitura dos arquivos de fase e preenche a matriz utilizada internamente para representar o cenário do jogo.
- `HandleBallBrickCollision`: detecta colisões entre a bola e os blocos, aplicando lógica de destruição, degradação de blocos, pontuação e acionamento de power-ups associados.
- `SpawnPowerUp`: cria power-ups a partir de blocos destruídos, posicionando-os no cenário e inicializando seu estado.
- `UpdatePowerUp`: atualiza o estado dos power-ups ativos, incluindo movimento, detecção de coleta e controle de duração de efeitos.
- `AtivaPowerUp`: aplica o efeito de um power-up no estado global do jogo, modificando propriedades como velocidade da bola, tamanho da plataforma, vidas ou score.
- `DesativaPowerUp`: reverte os efeitos de power-ups com duração limitada após expiração do tempo de efeito.
- `UpdateSingleBall`: atualiza a física de uma bola individual, incluindo movimento, colisões com paredes, blocos, plataforma e remoção ao sair da tela.
- `InitAudioSystem`: inicializa o sistema de áudio, carregando música principal e efeitos sonoros do jogo.

6. Implementação

O desenvolvimento foi realizado utilizando a linguagem C e a biblioteca Raylib, responsável pela criação da janela gráfica, renderização dos elementos visuais e captura das entradas do usuário.

A implementação do jogo foi estruturada em sistemas independentes que interagem entre si, incluindo gerenciamento de bolas, blocos, níveis, power-ups e áudio.

O sistema de bolas é responsável pela física principal do jogo, incluindo movimento, colisões com paredes, plataforma e blocos. Também implementa funcionalidades como multiball, aumento progressivo de velocidade e sincronização do estado da bola principal com a estrutura global do jogo. As colisões com blocos são processadas em conjunto com o sistema de nível, permitindo destruição progressiva e interação direta com power-ups.

Os blocos são organizados em uma matriz bidimensional de caracteres que representa o nível atual. Cada valor da matriz define o tipo e o estado do bloco, permitindo diferentes níveis de resistência e comportamento. A colisão entre bola e blocos é responsável por atualizar o estado do nível, aplicar pontuação e acionar a geração de power-ups quando necessário.

O sistema de power-ups é responsável por toda a lógica de recompensas do jogo. Ele inclui o carregamento de texturas, spawn de itens a partir de blocos destruídos, detecção

de colisão com o jogador, atualização de estado e aplicação de efeitos temporários ou permanentes. Alguns power-ups modificam diretamente propriedades globais do jogo, como velocidade da bola, tamanho da plataforma, vidas, pontuação e ativação de modos especiais como fireball e multiball. Também é responsável pela reversão de efeitos ao término da duração.

O sistema de níveis é responsável pelo carregamento das fases a partir de arquivos texto e pela preparação de mapas auxiliares de power-ups. Ele também controla a geração aleatória de power-ups por nível e verifica a condição de conclusão da fase, garantindo progressão entre estágios.

O sistema de áudio gerencia música e efeitos sonoros do jogo. Ele inicializa o dispositivo de áudio, carrega trilha sonora principal e efeitos como colisão de blocos, vitória e game over, além de manter a reprodução contínua da música durante o loop do jogo.

O sistema de pontuação atribui valores diferentes para cada tipo de bloco destruído e atualiza continuamente o score do jogador. O sistema de salvamento utiliza arquivos binários para persistir o estado completo da partida, permitindo retomada posterior exatamente do ponto onde o jogador parou.

O sistema de interface gráfica (UI) é responsável pela renderização de HUD, menus e feedback visual, integrando informações como vida, pontuação, estado do jogo e elementos auxiliares de jogabilidade.

O movimento da bola é calculado continuamente a partir de sua velocidade horizontal e vertical. A cada atualização do jogo são verificadas colisões com paredes, plataforma e blocos presentes na fase, garantindo comportamento consistente da física do jogo.

7. Instruções de Compilação

O projeto foi desenvolvido utilizando a biblioteca Raylib.

Após instalar a biblioteca, a compilação pode ser realizada a partir da raiz do projeto utilizando o seguinte comando:

```
gcc src/*.c -Iinclude -o bin/jogo \
-lraylib -lGL -lm -lpthread -ldl -lrt -lX11
```

Após a compilação, o jogo pode ser executado com:

```
./bin/jogo
```

8. Organização da Equipe

O desenvolvimento do projeto foi realizado em dupla. As atividades foram distribuídas de forma colaborativa entre os integrantes, abrangendo implementação, testes, correção de erros, elaboração da documentação e refinamento das funcionalidades do jogo.

Para facilitar o acompanhamento do progresso, foi criado um arquivo de tarefas (TODO.md) contendo as funcionalidades planejadas e um checklist de seus status. Essa organização permitiu distribuir responsabilidades de forma clara e acompanhar a evolução do projeto ao longo do tempo.

Embora cada integrante tenha assumido determinadas responsabilidades principais, as decisões de projeto e as etapas de validação foram realizadas conjuntamente, garantindo que ambos participassem de todas as fases do desenvolvimento.

9. Desafios e Aprendizados

Durante o desenvolvimento do projeto foram encontrados diversos desafios técnicos.

A implementação do sistema de colisões exigiu atenção especial para garantir comportamentos consistentes entre bola, blocos e plataforma. Além disso, foi necessário tratar adequadamente diferentes direções de impacto para evitar comportamentos incorretos durante a execução.

Um dos principais desafios do projeto foi a implementação do sistema de power-ups. Por se tratar de um sistema que altera diretamente múltiplas partes do jogo, ele exigiu modificações em diferentes módulos, incluindo a lógica da bola, plataforma, pontuação, estados globais do jogo e comportamento dos blocos. Isso gerou uma forte interdependência entre sistemas, tornando necessário um cuidado adicional para evitar acoplamento excessivo e garantir que os efeitos fossem aplicados e revertidos corretamente.

Outro desafio importante foi a modularização da aplicação. A separação das responsabilidades em diferentes arquivos exigiu um planejamento cuidadoso das estruturas compartilhadas e das interfaces entre módulos. Em alguns casos, foi necessário revisar a divisão inicial para evitar dependências circulares e garantir que cada módulo tivesse uma responsabilidade bem definida. Esse processo evidenciou a dificuldade prática de aplicar modularização em projetos de maior escala, especialmente quando diferentes sistemas precisam interagir constantemente.

A implementação dos sistemas de salvamento e ranking também exigiu a utilização de manipulação de arquivos em linguagem C, incluindo leitura, escrita e armazenamento persistente de informações. O sistema de salvamento representou um desafio significativo, pois exigiu a reorganização de parte da estrutura do projeto para permitir que os dados da partida fossem armazenados e restaurados corretamente.

Entre os principais aprendizados obtidos destacam-se a utilização de bibliotecas externas como o Raylib, a organização de projetos de maior porte, a manipulação de arquivos e o desenvolvimento de aplicações interativas em tempo real. Também foi possível compreender na prática os impactos de decisões de arquitetura de software, especialmente em relação à modularização e ao acoplamento entre sistemas.

10. Terefas Extras

Além dos requisitos básicos propostos para o trabalho, foram implementadas funcionalidades adicionais que ampliam a experiência do usuário:

- Efeitos sonoros implementados com o Raylib
- Power-ups adicionais, como vida extra, aumento da velocidade da plataforma e bola e aumento da pontuação

11. Créditos e Recursos Externos

A implementação do projeto utilizou referências externas e recursos de terceiros:

Durante o desenvolvimento foram consultadas fontes como StackOverflow, GeeksForGeeks e a documentação oficial da Raylib e da linguagem C. Funcionalidades não abordadas diretamente em aula foram baseadas em documentação e materiais externos devidamente referenciados no código-fonte.

Os efeitos sonoros e músicas utilizados no jogo possuem licença gratuita para uso, com os seguintes créditos:

- wyver9 — <https://opengameart.org/content/arcade-level-tracks-8-bitchiptune>
- ReyDros — <https://freesound.org/s/327054/>
- 0new4y — <https://opengameart.org/content/game-over-trumpet-sfx>
- spuipu — <https://opengameart.org/content/won-orchestral-winning-jingle>

12. Uso de Ia generativa

O uso de Ia foi estritamente para procura de material de apoio, ou seja, foi utilizada para encontrar conteúdo sobre tópicos utilizados. Seguem abaixo as Ias utilizadas para este propósito:

- ChatGPT - <https://chatgpt.com/>
- Gemini - <https://gemini.google.com/app?hl=pt-BR>
- Consensus - <https://consensus.app/>

13. Documentação

A documentação técnica do código foi gerada utilizando a ferramenta Doxygen. A documentação produzida inclui a descrição dos módulos, estruturas, arquivos e funções presentes no projeto.

Foram geradas versões em HTML e LaTeX, permitindo tanto a navegação interativa quanto a consulta em formato PDF.

Para gerar a documentação, execute o seguinte comando na raiz do projeto (onde se encontra o arquivo `Doxyfile`):

```
doxygen Doxyfile
```

A documentação em HTML pode ser visualizada no Linux através do comando:

```
xdg-open html/index.html
```

Também é possível gerar uma versão em PDF a partir dos arquivos LaTeX produzidos pelo Doxygen:

```
cd latex
make
```

Após a compilação, o documento PDF pode ser aberto com:

```
xdg-open refman.pdf
```

14. Conclusão

O desenvolvimento do Brick Breaker permitiu aplicar na prática diversos conceitos estudados ao longo da disciplina de Algoritmos e Programação. Durante a implementação foram utilizados conhecimentos relacionados a estruturas de dados, modularização, manipulação de arquivos, programação gráfica e organização de software.

Além de consolidar os conteúdos abordados em aula, o projeto proporcionou experiência no desenvolvimento de aplicações completas, desde a definição da arquitetura até a implementação e documentação do sistema.

A experiência adquirida durante o desenvolvimento contribuiu significativamente para o aprimoramento das habilidades de programação, resolução de problemas e trabalho em equipe.